

1) ADT Set Search

iterator inefficient?

d) HT w sch. } it is better for these DS to use
e) HT w o.a. } the hash functions.

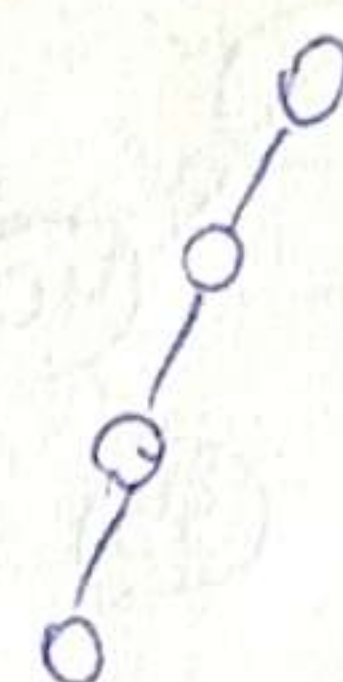
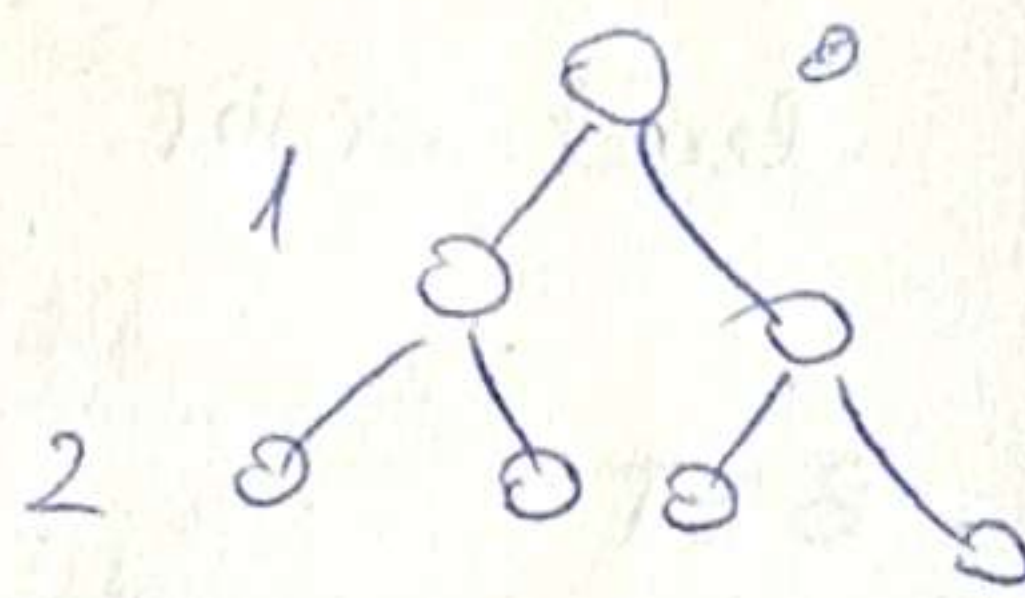
2) H = binary min. heap
> 1000 ≠ int.

a) true

~~b) true~~ b false.

c) true

(give ex.)



3) HT, linear probing

$$h(k, i) = (h'(k) + i) \% m$$

← take for each value

$m = 12$

in place of XX?

36. $h'(36) = 36 \% 12 = 0$

$h(36, 0) = (0 + 0) \% 12 = 0$ occupied

$h(36, 1) = (0 + 1) \% 12 = 1 \Rightarrow \boxed{f}$

11. $h'(11) = 11 \% 12 = 11$ XX

$h(11, 0) = (11 + 0) \% 12 = 11$ occupied

$h(11, 1) = 12 \% 12 = 0$ occupied

$h(11, 2) = 13 \% 12 = 1 \Rightarrow \boxed{e}$

XX

4) Inorder: ADHXHFVLI
Postorder: HXHD AVLFI

⇒ Right = ∅.

Postorder = reverse of Inorder



Inorder: ADHXHFVL

Postorder: HXHD AVLFI

In: ADHXH

Post: HXHDA

In: DXHXH

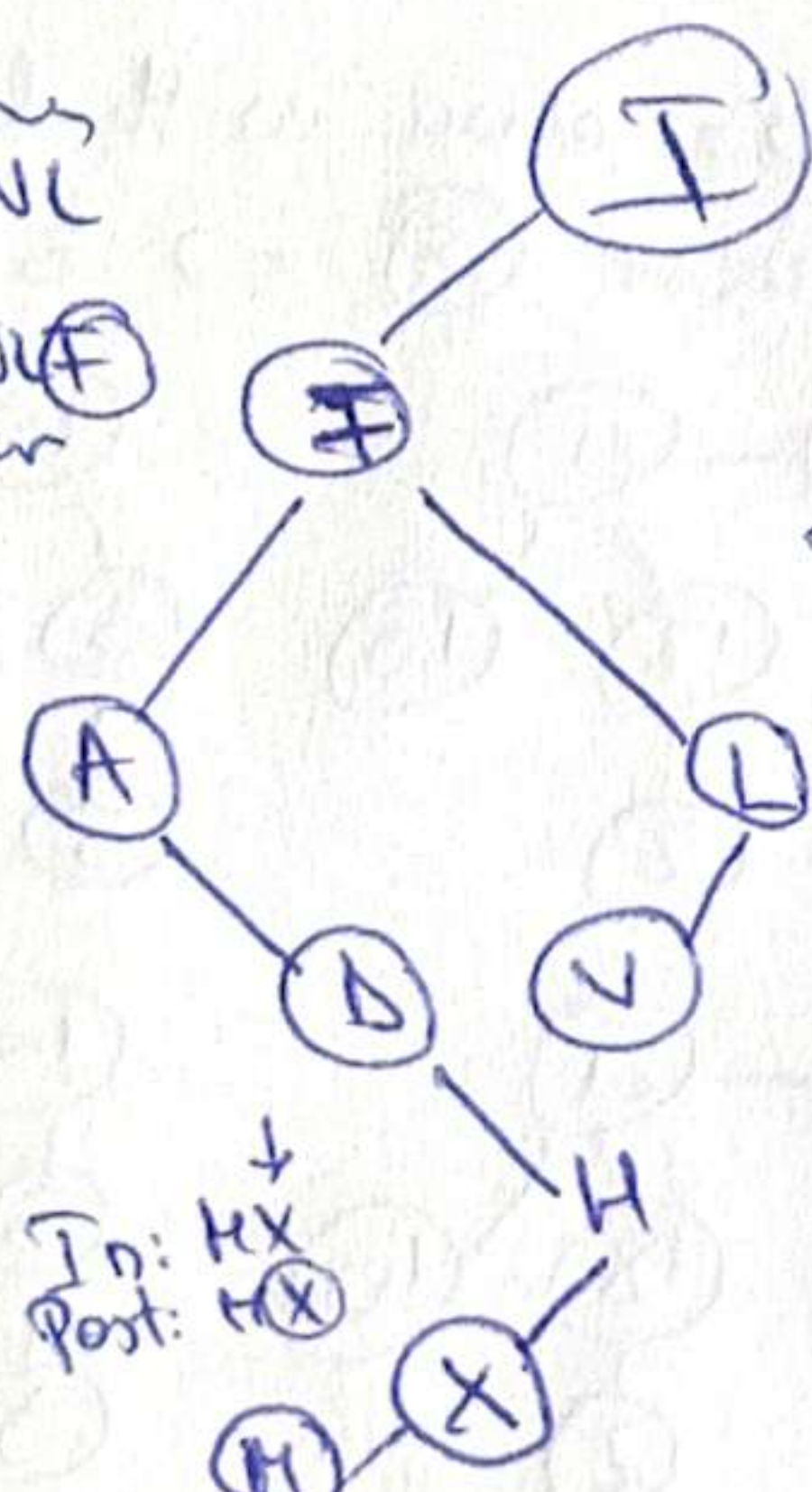
Post: HXHD

In: HXH

Post: HXH

In: HX

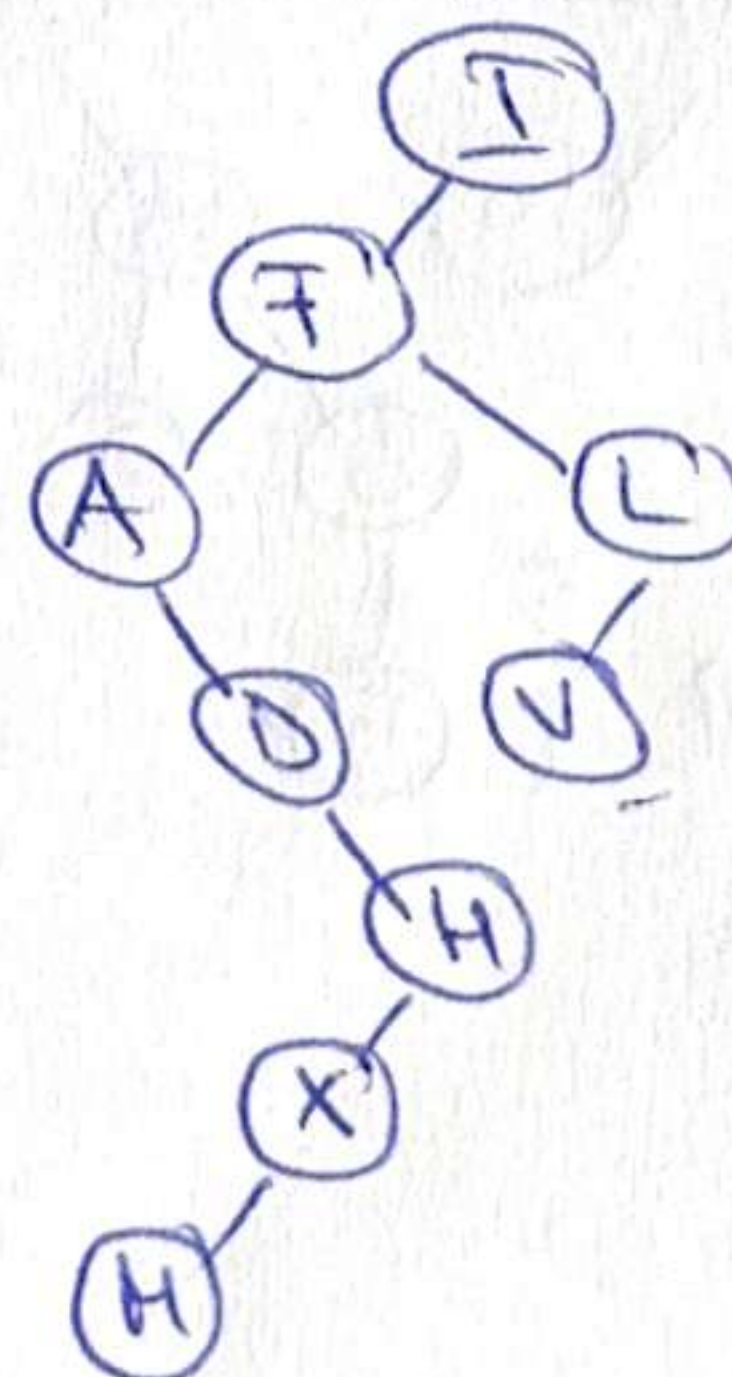
Post: HX



In: VL

Post: VL

In:



⇒ leaves:

~~V, H~~

V, H

f

c

B. 1) preorder: root-left-right

14 1 3 14 2 1 3 11 10 7 30 40 21 ✓

postorder: left-right-root

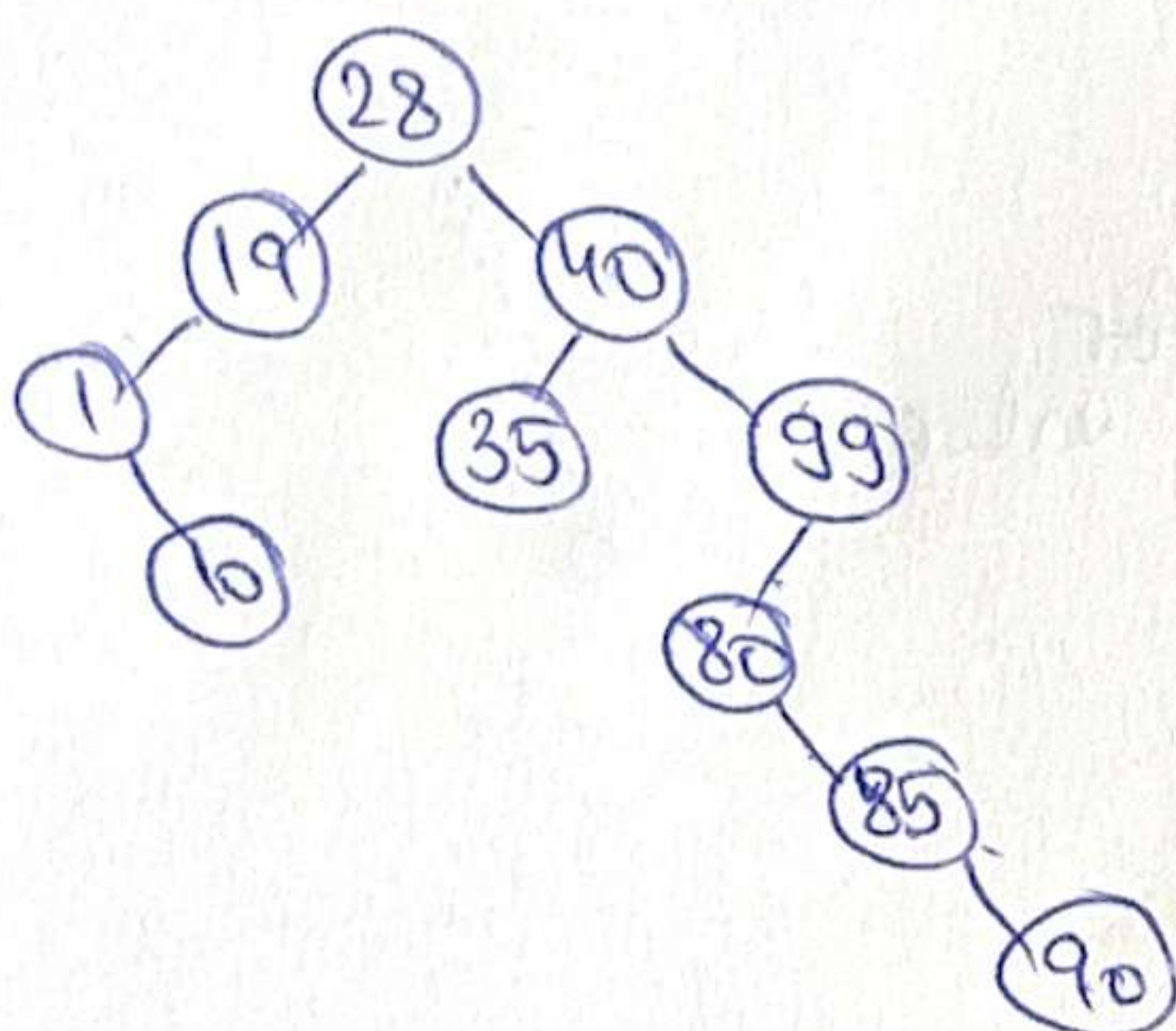
1 3 2 7 10 40 21 30 11 14

inorder: left-root-right

1 2 3 14 7 10 11 40 30 21

level order: 14 2 11 1 3 10 30 7 40 21

2) BST



height: 5

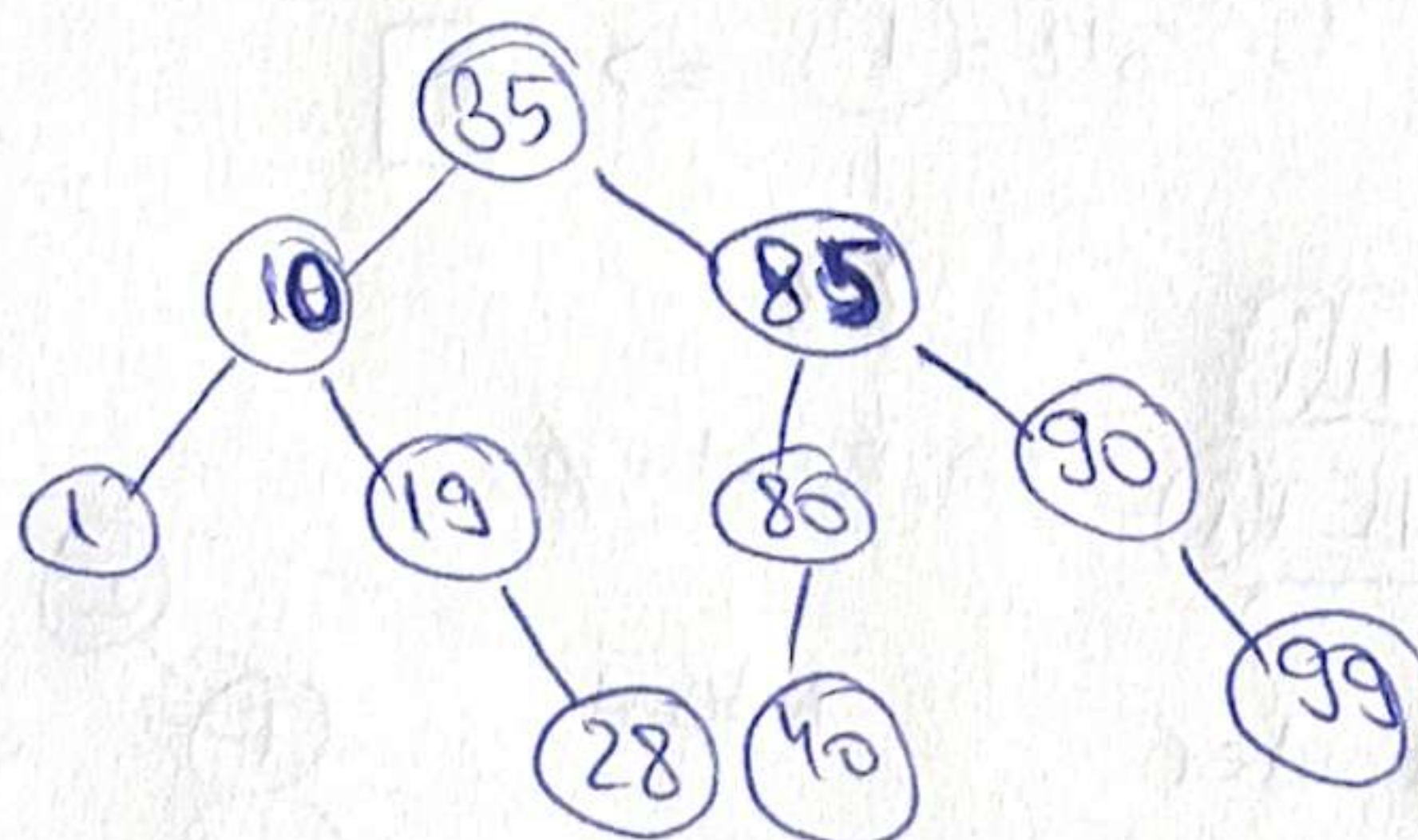
10 nodes

⇒ min. height tree.

$$\log_2 10 = 3$$

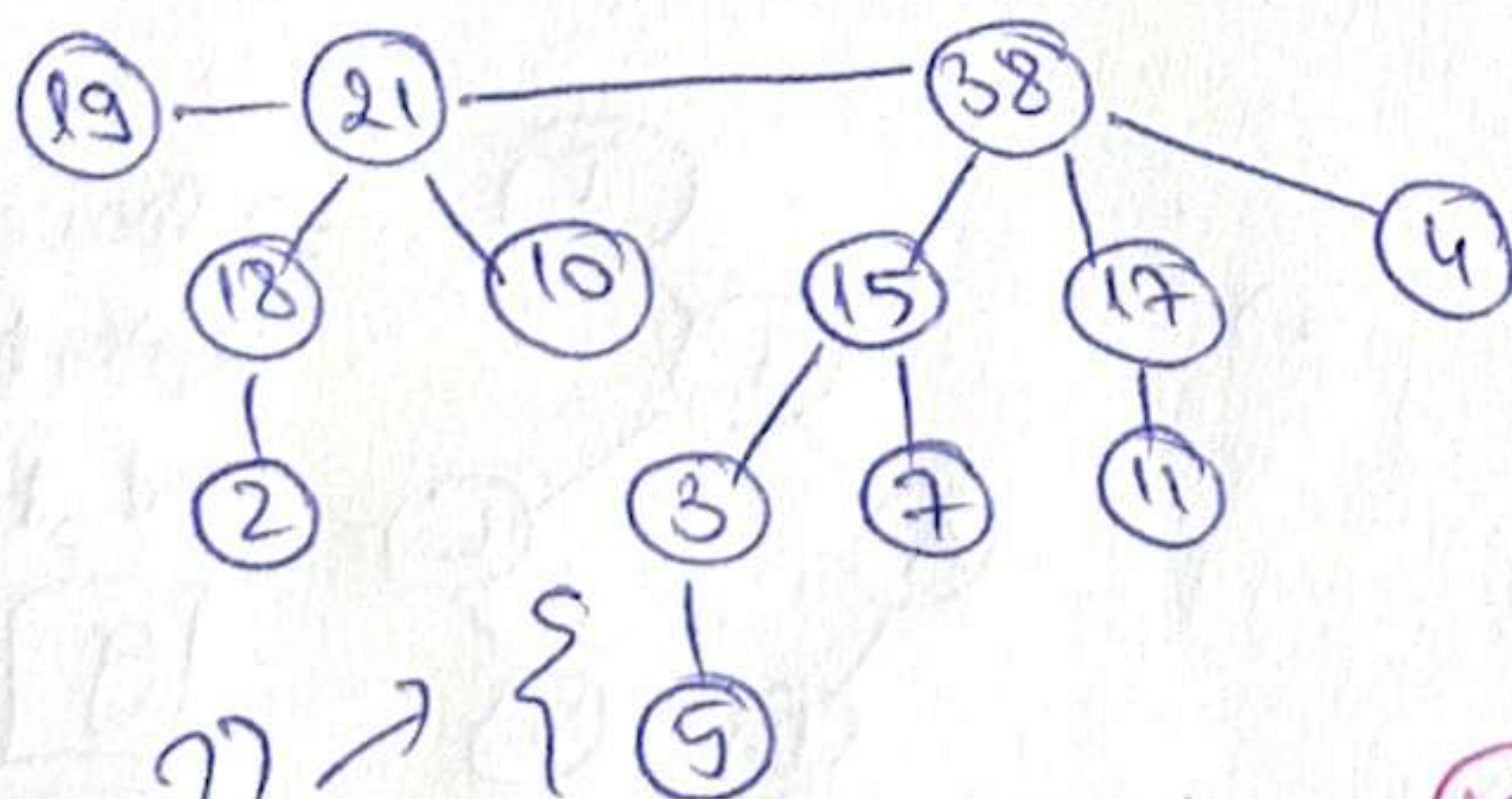
⇒ sort elem: 1, 10, 19, 28, 35, 40, 80, 85, 90, 99

⇒ level-order traversal:



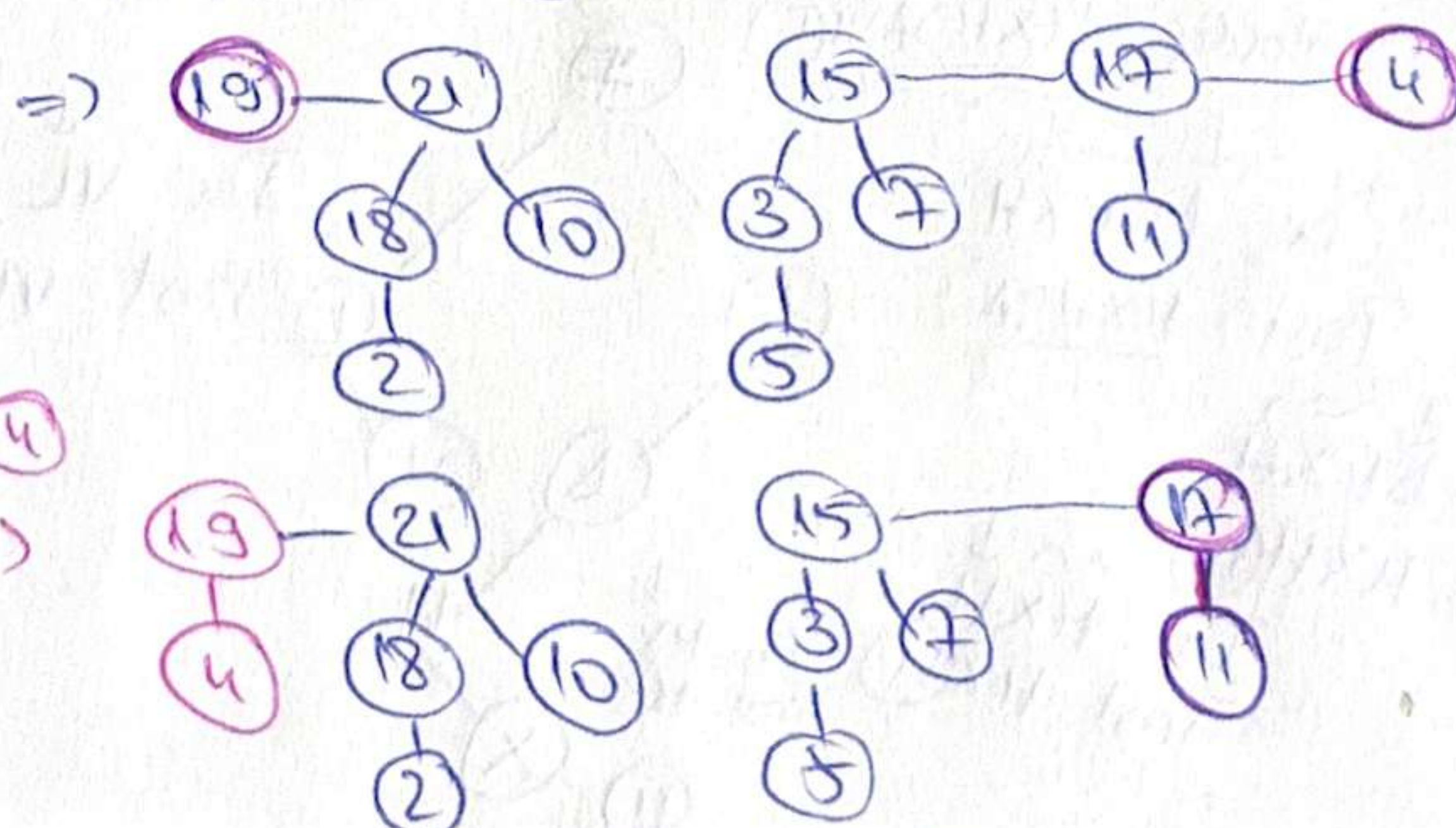
⇒ height: 3

3) Binomial heap.



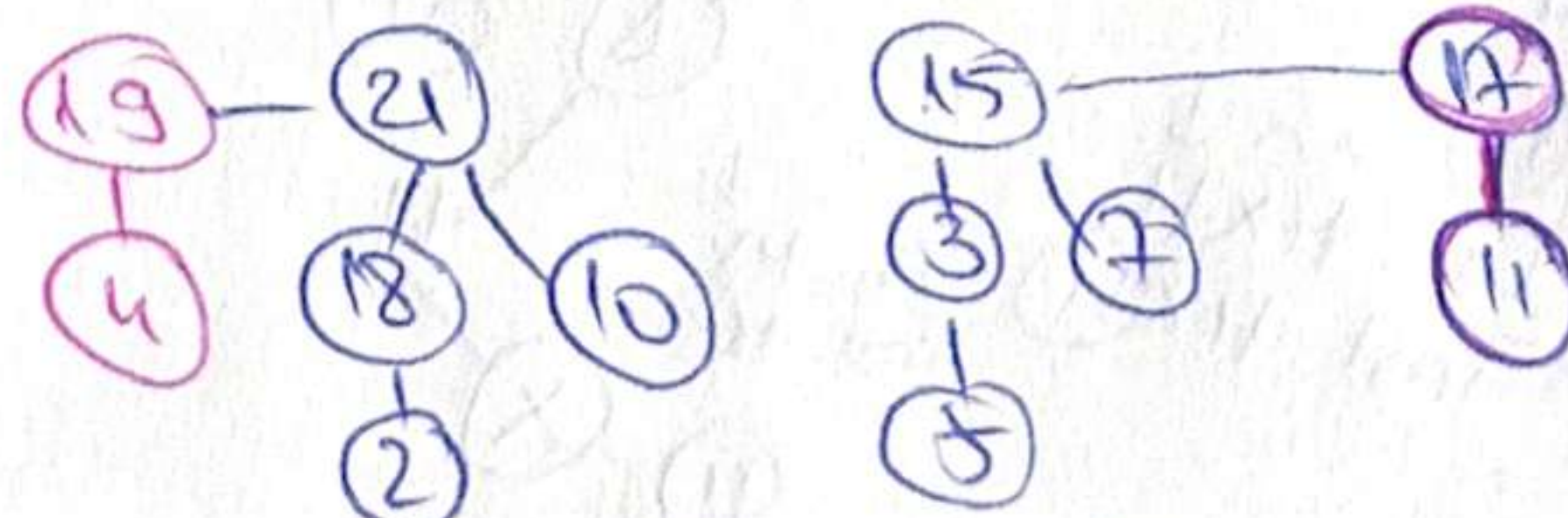
max-heap

max heap ⇒ elem with highest priority: 38.
⇒ remove 38 ⇒ 3 trees ⇒

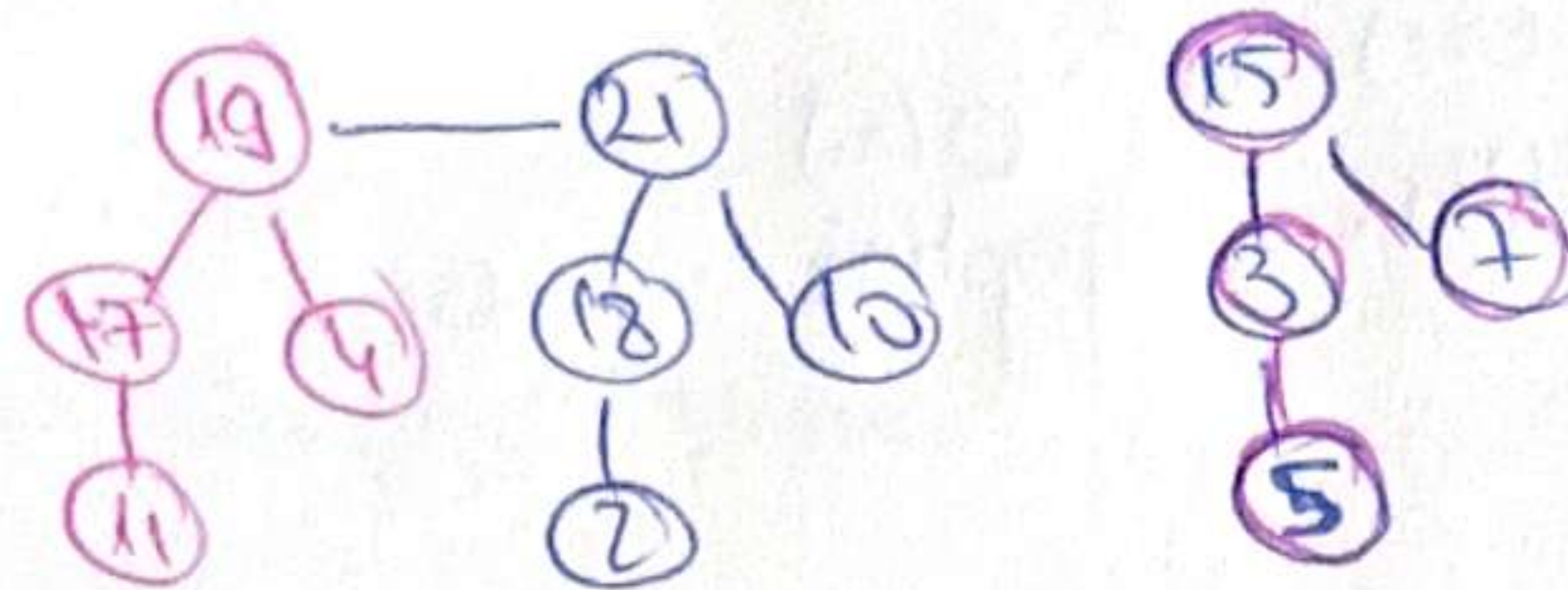


combine 19, 4

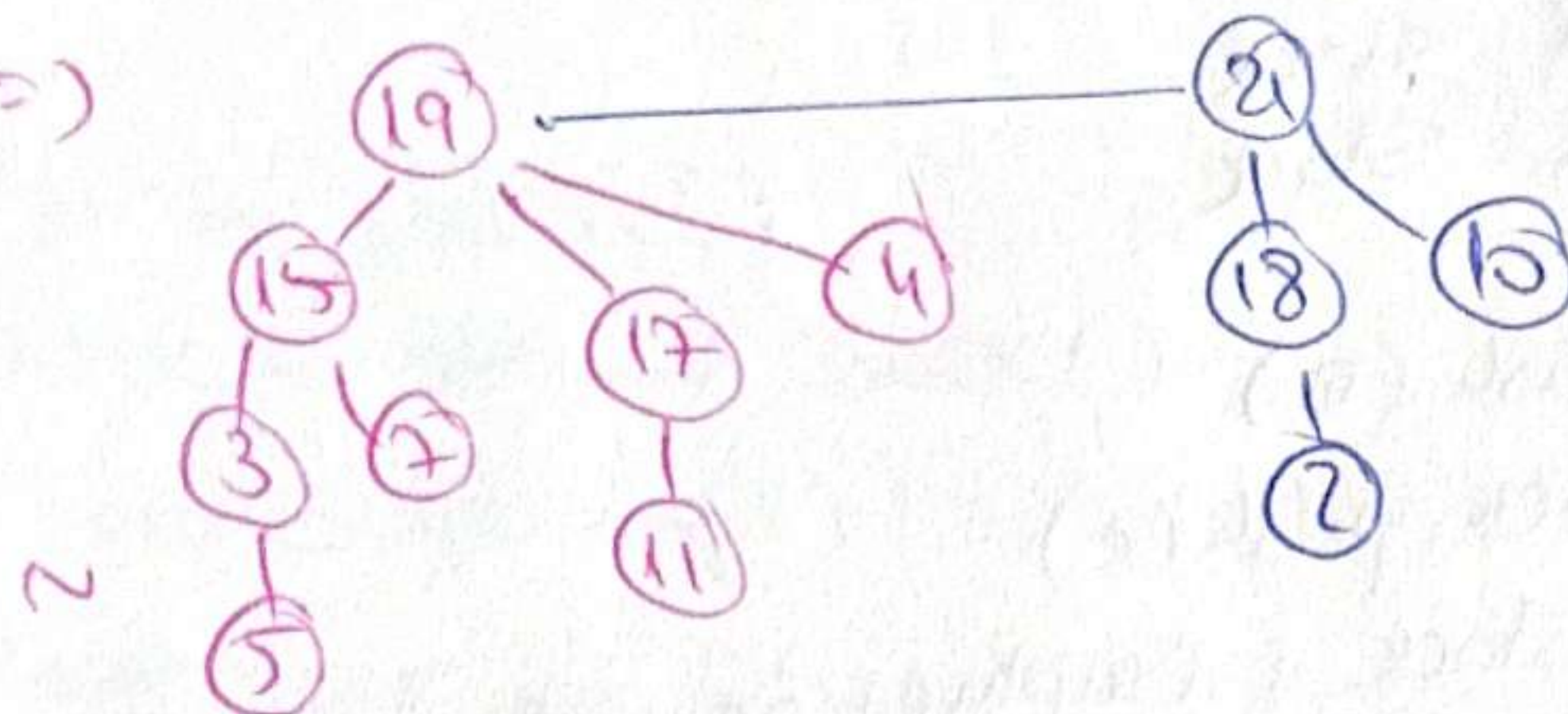
⇒



combine (19), (17) $\Rightarrow$



combine (19), (15) $\Rightarrow$



4) $m=11$, HT, coalesced chaining.

84 $\swarrow$ 98 $\searrow$ 29
 $\swarrow$ $\swarrow$ $\swarrow$

pos	0	1	2	3	4	5	6	7	8	9	10
elems	33	12	13	17	84	98	76	7	15	29	76
next	1	2	3	5	6	+6	-1	+10	6	8	9

firstEmpty: ~~0~~ ~~1~~ ~~2~~ ~~3~~ ~~4~~ ~~5~~ ~~6~~

$$h(7) = 7 \% 11 = 7$$

$$h(84) = 84 \% 11 = 4 \quad (7)$$

$$h(13) = 13 \% 11 = 2$$

$$h(33) = 33 \% 11 = 0$$

$$h(76) = 76 \% 11 = 10$$

$$h(29) = 29 \% 11 = 8 \quad 7$$

$$h(12) = 12 \% 11 = 1 \text{ occupied} \Rightarrow \text{firstEmpty} \Rightarrow 3$$

$$h(98) = 98 \% 11 = 10 \text{ occupied} \Rightarrow 3 \Rightarrow 5$$

remove 7 $\Rightarrow$ 33 12 13 - - - 84 98 29 76

e.

1) MaxStack $\rightarrow$ getMax $\theta(1)$, popMax $\theta(1)$

MaxStack:

mainStack: Stack
maxStack: Stack

function push(e):

mainStack.push(e)

if maxStack.isEmpty() or ~~max~~ e >= maxStack.top() then

maxStack.push(e)

else

maxStack.push(maxStack.top())

function pop():

if mainStack.isEmpty() then

@throw ...

mainStack.pop()

maxStack.pop()

function top():

if mainStack.isEmpty() then

@throw ...

~~return~~

top $\leftarrow$ mainStack.top()

function getMax():

if maxStack.isEmpty() then

@throw ...

getMax $\leftarrow$ maxStack.top()

function popMax():

if mainStack.isEmpty() then
 @throw...

targetMax ← getMax()

bufferStack ← ~~new~~ new Stack()

while mainStack.top() != targetMax do

 bufferStack.push(mainStack.top())
 pop()

pop()

while not bufferStack.isEmpty() do

 push(bufferStack.top())
 buf... .pop()